Slide 1 de 22

Bem-vindos ao Módulo de OOP!

Apresentação do módulo e metodologia

Neste módulo, vamos aprender **Programação Orientada a Objetos (POO)** com JavaScript. Ao final, vocês serão capazes de modelar entidades do mundo real em código e construir projetos completos usando esse paradigma.

- **Aulas 1–3**: Fundamentos (classes, métodos, encapsulamento)
- **Aula 4**: Herança, polimorfismo, static e prototype
- **Aulas 5–6**: Projeto guiado — To-Do List com OOP
- **Aulas 7–8**: Projeto em grupo — Jogo da Forca
- **Aula 9**: Apresentações e encerramento

A metodologia é prática: cada aula tem explicação com código ao vivo, estudos de caso reais e atividades de fixação. **Vocês vão codar junto comigo!**

Slide 2 de 22

🧪 Desafio Inicial

Sentindo a dor de viver sem classes

Antes de aprender OOP, vamos sentir **por que ela existe**. Usando apenas o que vocês já sabem, criem 5 funcionários de uma empresa:

```
const func1 = { nome: "Ana", cargo: "Dev", salario: 5000 };
const func2 = { nome: "Carlos", cargo: "QA", salario: 4500 };
const func3 = { nome: "Bia", cargoo: "Design", salario: 4000 };
const func4 = { nome: "Diego", cargo: "Dev", salario: 5200 };
const func5 = { nome: "Eva", funcao: "PM", salario: 6000 };
```

Perceberam os problemas? O `func3` tem um typo: `cargoo`. O `func5` usou `funcao` em vez de `cargo`. **Não existe padrão, não existe contrato.** E se precisássemos de 500 funcionários?

Slide 3 de 22

Os Problemas do Código Sem Estrutura

Limitações dos objetos literais

Usar apenas objetos literais para representar entidades traz vários problemas conforme o sistema cresce:

Problema	Consequência
Sem padrão de campos	Cada objeto pode ter campos diferentes — typos passam despercebidos
Sem validação	Qualquer valor pode ser atribuído (salário negativo? nome vazio?)
Código repetido	Mesma estrutura copiada dezenas de vezes
Sem comportamento	Funções ficam soltas, separadas dos dados que manipulam
Difícil manutenção	Mudar a estrutura exige alterar todos os objetos manualmente

Precisamos de algo que funcione como um **molde**: garanta que todos os objetos tenham os mesmos campos e comportamentos. É aí que entra a **Orientação a Objetos**.

Slide 4 de 22

Paradigma Funcional vs Orientado a Objetos

Duas formas de pensar em código

Existem diferentes formas de organizar código. Duas das mais comuns são o paradigma **funcional** e o **orientado a objetos**.

Aspecto	Funcional	Orientado a Objetos
Organização	Funções recebem dados por parâmetro	Dados e funções vivem juntos na entidade

Foco	Ações e transformações	Entidades e seus comportamentos
Analogia	Receita de bolo: ingredientes separados, instruções separadas	Padaria: o padeiro sabe seus ingredientes E sabe fazer o bolo
Quando brilha	Transformações de dados, pipelines	Entidades complexas que interagem entre si

```
// Abordagem FUNCIONAL — dados e funções separados
const cachorro = { nome: "Rex", energia: 100 };

function fazerCachorroLatir(dog) {
  console.log(`${dog.nome} diz: Au au!`);
}

function alimentarCachorro(dog, quantidade) {
  dog.energia += quantidade;
}

fazerCachorroLatir(cachorro);
alimentarCachorro(cachorro, 20);
```

No funcional, os dados (`cachorro`) ficam separados das funções que operam neles. No OO, tudo vai ficar junto — veremos isso em breve. **Nenhum é melhor que o outro**: são abordagens diferentes para problemas diferentes.

Slide 5 de 22

Verificação Rápida

Funcional vs OO

Qual é a principal diferença entre o paradigma funcional e o orientado a objetos?

✓ No funcional, dados e comportamentos ficam separados;

no OO, ficam juntos na entidade (Resposta Correta)

O funcional é mais rápido que o orientado a objetos

O orientado a objetos não usa funções

O funcional é mais moderno que o orientado a objetos

Explicação: Exato! No funcional, funções recebem dados por parâmetro. No OO, dados e comportamentos vivem juntos dentro da entidade (classe).

Slide 6 de 22

O que é Abstração?

Olhando para o mundo real e decidindo o que importa

Abstrair é olhar para algo do mundo real e selecionar apenas as características que importam **para o seu sistema**. O mesmo objeto real pode ter abstrações completamente diferentes dependendo do contexto.

Entidade: Carro	App do Uber	Sistema do Detran
placa	✅	✅
modelo	✅	✅
cor	✅	❌
localização GPS	✅	❌
chassi	❌	✅
renavam	❌	✅
proprietário	❌	✅
multas	❌	✅
potência do motor	❌	❌

tipo de óleo	❌	❌

O mesmo "Carro" tem atributos diferentes em cada sistema. **Isso é abstração**: decidir o que é relevante para o SEU contexto e ignorar o resto.

Slide 7 de 22

Tipo Abstrato de Dados (TAD)

Pensar antes de codar

Um **Tipo Abstrato de Dados (TAD)** é a ideia de definir uma estrutura com seus dados e operações **antes de implementar**. É pensar na modelagem antes de escrever código.

Pense no TAD como a **planta de um arquiteto**: antes de construir a casa, ele desenha a planta com cômodos, medidas e posições. Em programação, antes de codar, modelamos as entidades com seus atributos e comportamentos.

1. Identificar a entidade do mundo real (ex: Produto)
2. Definir os **dados** que ela armazena (ex: nome, preço, estoque)
3. Definir as **operações** que ela sabe fazer (ex: aplicar desconto, verificar disponibilidade)
4. Só depois: implementar em código

Esse processo de modelagem é o que vamos praticar com o **UML Simplificado** a seguir.

Slide 8 de 22

UML Simplificado

A linguagem visual de comunicação entre devs

UML (Unified Modeling Language) é uma linguagem visual para representar a estrutura de sistemas. Vamos usar uma versão simplificada: um retângulo dividido em 3 partes.

```
┌──────────────────────┐
│       Produto        │  ← Nome da Classe
├──────────────────────┤
```

```
│ nome                    │
│ preco                   │  ← Atributos (dados)
│ estoque                 │
│ categoria               │
├─────────────────────────┤
│ aplicarDesconto()       │
│ verificarEstoque()      │  ← Métodos (ações)
│ exibirInfo()            │
└─────────────────────────┘
```

UML é uma **linguagem de comunicação** entre desenvolvedores. Em empresas reais, o time modela em UML antes de escrever uma linha de código. É o **Figma do backend** — um rascunho visual que todo mundo entende.

Neste módulo, usamos a versão simplificada: sem tipos de dados nem indicadores de visibilidade. O foco é no hábito de **modelar antes de codar**.

Slide 9 de 22

Verificação Rápida

Abstração e UML

Se você está criando um app de delivery de comida, qual atributo NÃO faz sentido para a classe 'Restaurante'?

- nome
- endereco
- ✓ CNPJ do contador do dono (Resposta Correta)
- horarioFuncionamento

Explicação: Isso! O CNPJ do contador não é relevante para um app de delivery. Abstração é justamente isso: incluir apenas o que importa para o contexto.

Slide 10 de 22

Sua Primeira Classe em JavaScript

class, constructor e this

Agora vamos transformar o UML em código real. Uma **classe** é o molde. O **constructor** é o método que roda automaticamente quando criamos um objeto. O **this** referencia a instância que está sendo criada.

```
class Produto {
  constructor(nome = "Sem nome", preco = 0, estoque = 0) {
    this.nome = nome;
    this.preco = preco;
    this.estoque = estoque;
  }
}

// Criando instâncias (objetos concretos a partir do molde)
const cafe = new Produto("Café", 12.90, 50);
const acucar = new Produto("Açúcar", 5.50, 100);
const biscoito = new Produto(); // usa todos os valores padrão

console.log(cafe.nome);      // "Café"
console.log(cafe.preco);     // 12.90
console.log(biscoito.nome);  // "Sem nome"
```

- `class Produto {}` — declara o molde (a fôrma do bolo)
- `constructor()` — roda automaticamente quando fazemos `new`
- `this.nome = nome` — atribui o valor à instância sendo criada
- `new Produto(...)` — cria um objeto concreto a partir do molde
- Valores padrão (`= "Sem nome"`) protegem contra `undefined`

Slide 11 de 22

Entendendo o this

O this é 'eu mesmo' dentro do objeto

O `this` é uma das palavras mais importantes (e confusas) do JavaScript. Dentro de uma classe, ele se refere à **instância específica** que está sendo criada ou utilizada.

```
class Pessoa {
  constructor(nome) {
    // Quando fazemos new Pessoa("Ana"),
    // this é o objeto "Ana" sendo criado
    this.nome = nome;
    console.log(this); // mostra o objeto nascendo!
  }
}

const ana = new Pessoa("Ana");
// Output: Pessoa { nome: 'Ana' }

const carlos = new Pessoa("Carlos");
// Output: Pessoa { nome: 'Carlos' }

// Cada instância tem seu próprio this
console.log(ana.nome);    // "Ana"
console.log(carlos.nome); // "Carlos"
```

Quando a classe tem 50 instâncias, cada `this` aponta para **o seu próprio objeto**. O `this` do `ana` é diferente do `this` do `carlos`.

Slide 12 de 22

Instância vs Classe

A diferença crucial

Essa diferença gera confusão em quem está começando. Vamos deixar bem claro:

Conceito	O que é	Analogia
Classe	O molde/modelo/planta	A fôrma do bolo

Instância	Um objeto concreto criado a partir do molde	Um bolo feito com a fôrma
new	O operador que cria uma instância	O ato de usar a fôrma para fazer um bolo
constructor	O código que inicializa os atributos	A receita seguida ao fazer o bolo

```
const cafe = new Produto("Café", 12.90, 50);
const acucar = new Produto("Açúcar", 5.50, 100);

// Cada instância é independente
cafe.preco = 14.90;  // Só muda o café
console.log(acucar.preco); // 5.50 — inalterado!

// Posso adicionar atributo a UM objeto
cafe.categoria = "Bebidas";
console.log(cafe.categoria);   // "Bebidas"
console.log(acucar.categoria); // undefined! — não existe na classe
```

Atenção: adicionar um atributo a um objeto (`cafe.categoria`) NÃO altera a classe. Os próximos objetos criados com `new Produto()` não terão `categoria` .

Slide 13 de 22

Verificação Rápida

Classes e instâncias

O que acontece ao executar: `const p = new Produto(); p.cor = 'azul'; const q = new Produto(); console.log(q.cor);`?

Imprime 'azul'

✓ Imprime undefined (Resposta Correta)

Dá erro

Imprime null

Explicação: Correto! Adicionar `cor` ao objeto `p` não muda a classe Produto. O objeto `q` foi criado a partir do molde original, que não tem `cor`.

Slide 14 de 22

instanceof — Verificando a Origem

Diferença entre objeto literal e instância de classe

Como saber se um objeto veio de uma classe específica? Usamos o operador `instanceof`:

```
class Produto {
  constructor(nome, preco) {
    this.nome = nome;
    this.preco = preco;
  }
}

const cafe = new Produto("Café", 12.90);
const objetoSolto = { nome: "Café", preco: 12.90 };

console.log(cafe instanceof Produto);        // true ✅
console.log(objetoSolto instanceof Produto); // false ❌

// Parecem iguais, mas têm origens diferentes!
console.log(typeof cafe);        // "object"
console.log(typeof objetoSolto); // "object"
```

O `typeof` diz que ambos são `"object"`, mas o `instanceof` revela

a verdade: o `cafe` veio de um molde (`Produto`), o `objetoSolto` é avulso. A instância tem **contrato** — ela segue a estrutura da classe.

Slide 15 de 22

Caso de Uso Real — Como Apps Modelam Entidades

Exemplos do mundo real

Todos os apps que vocês usam no dia a dia modelam entidades com classes. Vamos ver alguns exemplos:

App	Algumas Classes Prováveis
iFood	Restaurante, Prato, Pedido, Entregador, Endereco, Avaliacao
Mercado Livre	Produto, Vendedor, Comprador, Avaliacao, Carrinho, Pagamento
Spotify	Musica, Playlist, Usuario, Album, Artista, Podcast
Instagram	Post, Usuario, Comentario, Story, Mensagem, Notificacao
Uber	Motorista, Passageiro, Viagem, Veiculo, Pagamento, Avaliacao

Cada classe tem atributos que fazem sentido **para aquele negócio**. É a abstração em ação: um `Produto` no Mercado Livre tem `vendedor` e `frete` , mas no iFood o `Prato` tem `tempoPreparo` e `restaurante` .

Slide 16 de 22

Estudo de Caso — Classe Carro

Do UML ao código

Vamos praticar o fluxo completo: **UML → Código → Instâncias**. Contexto: um app de revenda de veículos.

```
┌──────────────────────┐
│        Carro         │
```

```
├──────────────────────┤
│ marca                │
│ modelo               │
│ ano                  │
│ preco                │
│ km                   │
│ vendido              │
├──────────────────────┤
│ (métodos na aula 2) │
└──────────────────────┘
```

```
class Carro {
  constructor(marca = "N/A", modelo = "N/A", ano = 2024, preco
= 0, km = 0) {
    this.marca = marca;
    this.modelo = modelo;
    this.ano = ano;
    this.preco = preco;
    this.km = km;
    this.vendido = false;
  }
}

const carro1 = new Carro("Toyota", "Corolla", 2022, 120000,
35000);
const carro2 = new Carro("Honda", "Civic", 2023, 135000,
12000);
const carro3 = new Carro(); // todos os padrões

console.log(carro1.marca);   // "Toyota"
console.log(carro1.vendido); // false
console.log(carro3.modelo);  // "N/A"
```

Slide 17 de 22

Estudo de Caso — Classe Cachorro

Mais um exemplo de classe completa

```
┌──────────────────────┐
│      Cachorro        │
├──────────────────────┤
│ nome                 │
│ raca                 │
│ idade                │
│ peso                 │
│ vacinado             │
├──────────────────────┤
│ (métodos na aula 2)  │
└──────────────────────┘
```

```javascript
class Cachorro {
  constructor(nome = "Sem nome", raca = "SRD", idade = 0, peso
= 0) {
    this.nome = nome;
    this.raca = raca;
    this.idade = idade;
    this.peso = peso;
    this.vacinado = false;
  }
}

const rex = new Cachorro("Rex", "Pastor Alemão", 3, 35);
const mel = new Cachorro("Mel", "Golden Retriever", 2, 28);
const vira = new Cachorro("Bolinha"); // SRD, idade 0, peso 0

console.log(rex.raca);    // "Pastor Alemão"
console.log(vira.raca);   // "SRD"
console.log(mel.vacinado); // false
```

Perceba o padrão: `raca = "SRD"` como valor padrão faz sentido no contexto — se não informar a raça, é "Sem Raça Definida". Os valores padrão devem ter **significado no domínio**.

Slide 18 de 22

Verificação Rápida

Construindo classes

Qual é o resultado de: `class X { constructor(a = 10) { this.a = a; } } const obj = new X(); console.log(obj.a);`?

- undefined
- ✓ 10 (Resposta Correta)
- null
- Erro — precisa passar argumento

Explicação: Correto! O valor padrão `a = 10` é usado quando nenhum argumento é passado. O `this.a` recebe 10.

Slide 19 de 22

Exercício Interativo — Crie sua Classe

Hora de praticar!

Agora é sua vez! Vamos pensar juntos: **se vocês fossem criar um clone do Spotify**, quais classes precisariam? Que atributos cada uma teria?

Pense em pelo menos 3 classes com 4+ atributos cada. Lembre-se: **abstrair é escolher o que importa** para o seu sistema.

```
// Exemplo para começar:
class Musica {
  constructor(titulo = "", artista = "", duracao = 0, album =
"") {
    this.titulo = titulo;
    this.artista = artista;
    this.duracao = duracao; // em segundos
    this.album = album;
    this.curtida = false;
  }
}

// Agora modele: Playlist, Usuario, Album...
// Quais atributos cada um precisa?
// Desenhe o UML simplificado primeiro!
```

Slide 20 de 22

Atividade de Fixação 1

Formas geométricas — classes e atributos

Atividade prática (escopo fechado): Crie classes para formas geométricas. Essas classes serão reutilizadas nas próximas aulas!

1. Desenhe o **UML simplificado** para pelo menos 3 formas: `Circulo`, `Retangulo` e `Triangulo`
2. Implemente as classes em JavaScript com **atributos no constructor** e **valores padrão**
3. Crie **pelo menos 2 instâncias** de cada forma
4. Exiba os atributos de cada instância no console

```
// Exemplo de como começar:
class Circulo {
  constructor(raio = 1) {
    this.raio = raio;
```

```
  }
}

class Retangulo {
  constructor(base = 1, altura = 1) {
    // Seu código aqui
  }
}

class Triangulo {
  constructor(/* quais parâmetros? */) {
    // Seu código aqui
  }
}

// Crie instâncias e teste!
const c1 = new Circulo(5);
const c2 = new Circulo(10);
console.log(c1.raio); // 5
console.log(c2.raio); // 10
```

Guarde esse código! Vamos adicionar métodos (aula 2) e encapsulamento (aula 3) nessas mesmas classes.

Slide 21 de 22

Lição de Casa — Aula 1

Sistema de biblioteca

Para entregar: Modele um sistema de biblioteca com 3 classes. Cada classe deve ter pelo menos 4 atributos com valores padrão.

```
┌──────────────────────┐   ┌──────────────────────┐
┌──────────────────────┐
│       Livro          │   │       Autor          │   │       Leit
├──────────────────────┤   ├──────────────────────┤   ├──────────────
```

```
│ titulo                  │    │ nome                    │    │ nome
│ autor                   │    │ nacionalidade           │    │ idade
│ paginas                 │    │ anoNascimento           │    │ email
│ isbn                    │    │ livrosPublicados        │    │ livrosLido
├─────────────────────────┤    ├─────────────────────────┤    ├──────────────
│ (métodos na aula 2) │    │ (métodos na aula 2) │    │ (métodos
na aula 2) │
└─────────────────────────┘    └─────────────────────────┘    └──────────────
```

- Desenhe o UML simplificado (foto de papel vale!)
- Implemente as 3 classes com constructor e valores padrão
- Crie pelo menos 2 instâncias de cada e teste no console
- Pense: que métodos cada classe poderia ter? (não precisa implementar ainda)

Slide 22 de 22

Resumo da Aula 1

O que aprendemos hoje

Conceito	Definição Rápida
Abstração	Selecionar apenas o que importa para o contexto do sistema
TAD	Definir estrutura e operações antes de implementar
UML Simplificado	Diagrama visual: Nome | Atributos | Métodos
Classe	O molde/modelo que define a estrutura
Instância	Um objeto concreto criado com new a partir da classe
constructor	Método que roda automaticamente no new
this	Referência à instância atual ('eu mesmo')
Valores padrão	Parâmetros com valor default no constructor
instanceof	Operador que verifica se um objeto veio de uma classe

Na **próxima aula**: as classes vão ganhar **comportamento** — vamos aprender sobre **métodos**!

Total de 22 slides